

1-2机器学习的基本概念（深度学习）- 整理版

2026年9月2日 17:21

背景：线性模型的“模型偏差”（Model Bias）

- 第一讲中的线性模型 $y = b + wx$ 是一条直线，无法表示折线或曲线（如现实中的气温变化、股票走势）。
- 这种因模型本身表达能力不足造成的误差，称为 **模型偏差（Model Bias）**。
- 解决方法：引入**非线性**，通过叠加多个基础函数来拟合任意形状的曲线。

Step 1：写出带有未知参数的函数（引入非线性）

① 基本砖块：Sigmoid 函数

- 公式：

Sigmoid Function

$$y = c \frac{1}{1 + e^{-(b+wx_1)}}$$

$$= c \operatorname{sigmoid}(b + wx_1)$$



图像为平滑的“S”形曲线。

- 通过调整三个参数，可改变其形状：
- w （权重）：控制坡度（陡峭或平缓）
- b （偏置）：控制左右平移
- c （幅度）：控制高度（正为凸起，负为凹陷）
- 一个 Sigmoid 相当于一个“小山坡”。

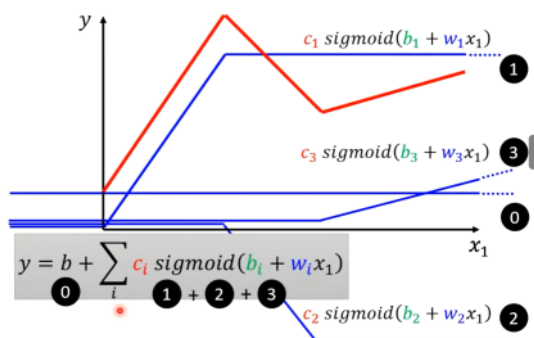
② 用多个 Sigmoid 叠加出任意曲线

- 任何分段线性曲线（或平滑曲线）都可以看作：

$$\text{红色曲线} = \text{常数} + \sum_i c_i \cdot \sigma(b_i + w_i x)$$

即 常数项 + 多个不同位置/坡度/高矮的 Sigmoid 之和。

red curve = sum of a set of  + constant



③ 引入多个特征（多维输入）

- 如果输入不只一个特征（比如前 7 天数据），则每个 Sigmoid 的输入变为所有特征的加权和：

$$r_i = b_i + \sum_j w_{ij} x_j$$

最终预测值为：

$$y = b + \sum_i c_i \cdot \sigma(r_i)$$

New Model: More Features

$$y = b + wx_1$$

$$y = b + \sum_i c_i \text{sigmoid}(b_i + w_i x_1)$$

$$y = b + \sum_j w_j x_j$$

$$y = b + \sum_i c_i \text{sigmoid}\left(b_i + \sum_j w_{ij} x_j\right)$$

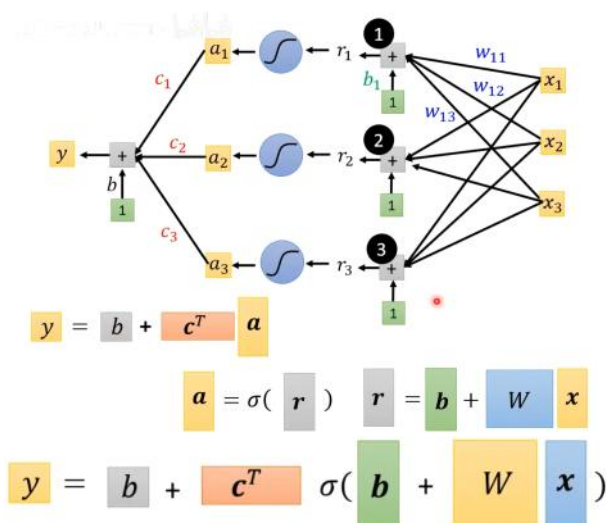
- 用向量和矩阵表示，可写成紧凑形式（即一个神经网络层）。

$$y = b + \sum_i c_i \text{sigmoid}\left(b_i + \sum_j w_{ij} x_j\right) \quad \begin{matrix} i: 1,2,3 \\ j: 1,2,3 \end{matrix}$$

$$\begin{aligned} r_1 &= b_1 + w_{11}x_1 + w_{12}x_2 + w_{13}x_3 \\ r_2 &= b_2 + w_{21}x_1 + w_{22}x_2 + w_{23}x_3 \\ r_3 &= b_3 + w_{31}x_1 + w_{32}x_2 + w_{33}x_3 \end{aligned}$$

$$\begin{bmatrix} r_1 \\ r_2 \\ r_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} + \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

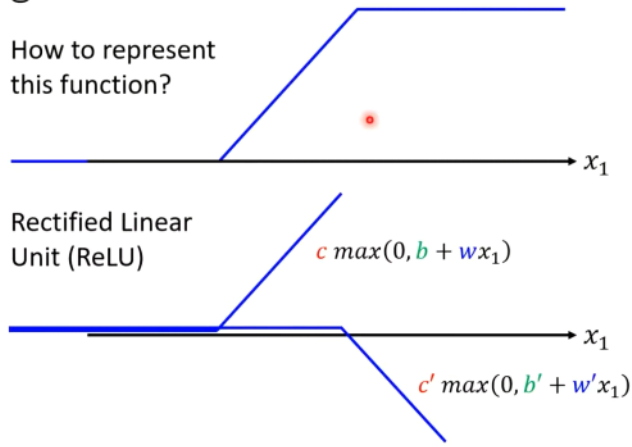
$$\mathbf{r} = \mathbf{b} + \mathbf{W} \mathbf{x}$$



④ 另一种常用砖块：ReLU

- ReLU 函数： $\text{ReLU}(x) = \max(0, x)$ ，计算极快。
- 两个 ReLU 可以合成一个 Hard Sigmoid（即折线近似的 Sigmoid）。
- 现代深度学习普遍使用 ReLU 及其变体作为激活函数。

Sigmoid → ReLU



$$y = b + \sum_i c_i \max\left(0, b_i + \sum_j w_{ij} x_j\right)$$

Step 2: 定义损失函数 (Loss)

- 和线性模型相同，用训练数据计算预测值 y 与真实值 $\hat{y}$ 的差距。
- 常用误差：
- MAE: $e = |y - \hat{y}|$
- MSE: $e = (y - \hat{y})^2$
- 损失 L 为所有样本误差的平均值，目标是**最小化** L 。

新增概念：批量 (Batch)

- 当数据量很大时，不一次性计算所有数据，而是分成**小批量 (Batch)**。
- 每处理完一个 Batch，就计算 Loss 并**更新**一次参数。
- 遍历全部 Batch 一次**，称为一个 **Epoch (回合)**。
- 使用 Batch 的好处：节省内存，且小批量的随机梯度有助于跳出鞍点。

Optimization of New Model

$$\theta^* = \arg \min_{\theta} L$$

➤ (Randomly) Pick initial values θ^0

➤ Compute gradient $g = \nabla L^1(\theta^0)$

update $\theta^1 \leftarrow \theta^0 - \eta g$

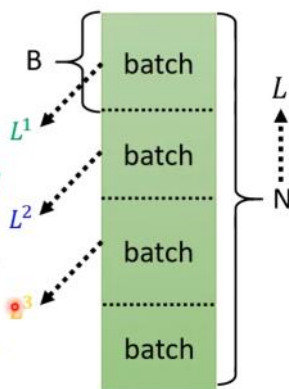
➤ Compute gradient $g = \nabla L^2(\theta^1)$

update $\theta^2 \leftarrow \theta^1 - \eta g$

➤ Compute gradient $g = \nabla L^3(\theta^2)$

update $\theta^3 \leftarrow \theta^2 - \eta g$

1 epoch = see all the batches once



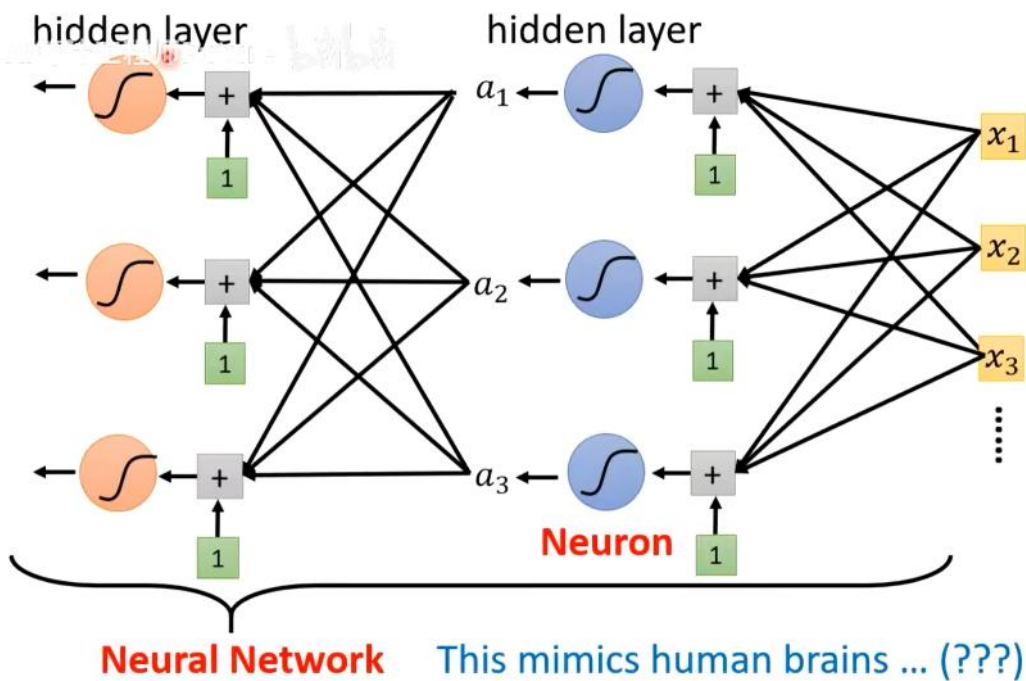
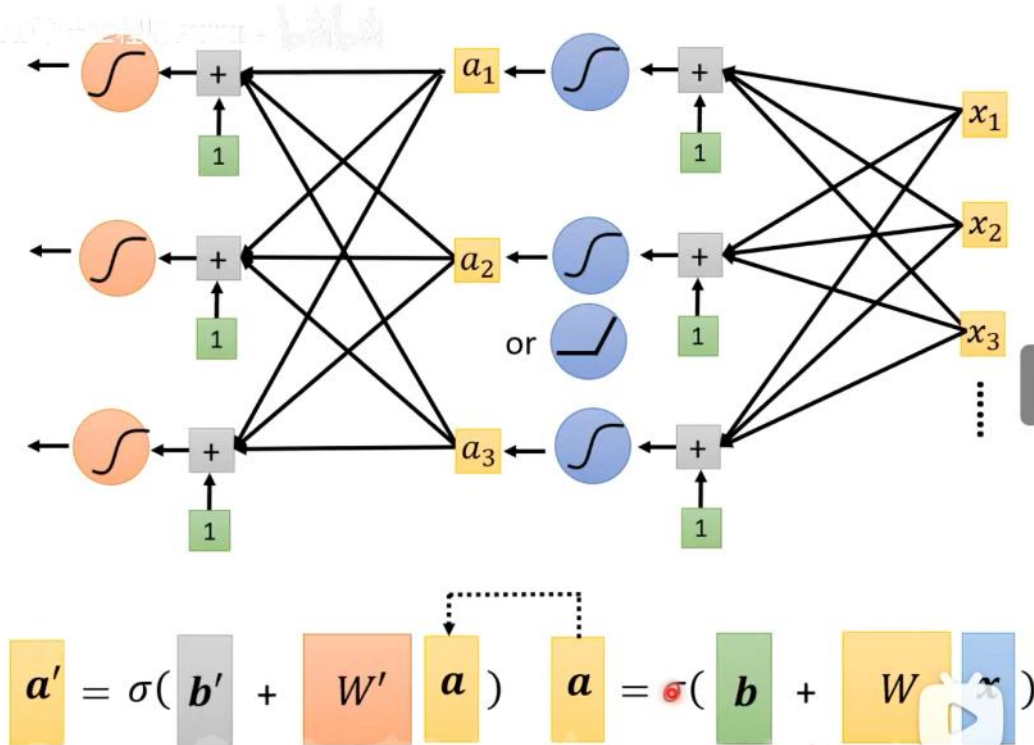
Step 3: 优化 (Optimization) —— 梯度下降

- 与线性模型相同，计算 Loss 对每个参数 (w , b , c 等) 的偏导数。
- 沿梯度反方向**更新参数**，步长由学习率 η 控制。
- 重复迭代直到 Loss **收敛**。
- 在实践中，深度学习框架会自动计算梯度并更新参数。

从“单层”到“多层”——深度学习的诞生

- 一个 Sigmoid (或 ReLU) 可以看作一个神经元。
- 将多个神经元组成一层**隐藏层 (Hidden Layer)**，该层的输出作为下一层的输入，层层堆叠，形成多层神经网络。
- 因为层数很深，所以称为 Deep Learning (深度学习)。
- 结构示意：

输入层 → 隐藏层1 → 隐藏层2 → ... → 输出层



Many layers means **Deep** ➡ Deep Learning

总结与延伸

- 本课核心：用非线性激活函数 (Sigmoid/ReLU) 叠加，克服线性模型的“直”缺陷，使模型能够拟合任意复杂曲线。
- 通过引入多个特征和多个神经元，并堆叠多层网络，我们构建了深度神经网络的雏形。

- **激活函数 (Activation Function)** 是神经网络非线性的来源，常见的有 **Sigmoid**、**ReLU**、**Tanh** 等。
- 后续课程将进一步探讨如何**选择层数、神经元个数**，以及**过拟合、梯度消失**等更深入的问题。

一句话概括：深度学习 = 多层非线性变换 (**Sigmoid/ReLU 叠加** + **梯度下降优化**)，让机器学会拟合任何复杂的输入-输出关系。